# Assignment-19.1

**HT.NO:2303A51912**

**Task 1 – Python to Java Conversion**

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

CODE:

task_1.py > ...

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)

s = Student("Akram", 21)
s.display()
```

```
task_1.java > Student
class Student {
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Method
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }

    Run | Debug
    public static void main(String[] args) {
        Student s = new Student(name: "Akram", age: 21);
        s.display();
    }
}
```

**Task 2 – C++ to Python Function Conversion**

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:

• A correct Python implementation equivalent to the original C++ factorial function.

**CODE**:

```python
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

print(factorial(5))
```

```cpp
#include <iostream>
using namespace std;

int factorial(int n) {
    if(n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

int main() {
    cout << factorial(5);
    return 0;
}
```

**Task 3 – SQL to Pandas Query**

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

CODE:

```python
import pandas as pd

data = {
    'name': ['John', 'Alice', 'Bob', 'Emma'],
    'salary': [45000, 60000, 75000, 40000]
}

df = pd.DataFrame(data)

result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

**Task 4 – Real-Time Application: Multi-Language Snippet**

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

| | name | salary |
|---|---|---|
| 1 | Alice | 60000 |
| 2 | Bob | 75000 |

Converting a Python web scraping snippet into JavaScript for browser automation.

• Translating a Java method into C# for enterprise applications.

Instructions:

• Prompt AI to translate code between two chosen languages.

• Test both implementations to ensure they produce the same results.

• Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

• Working equivalent code snippets in at least two different programming languages

CODE:

```
task_1.py   task_1.java   task_2.c++   task_3.sql   task_4.py
task_4.py > ...
1   import requests
2   from bs4 import BeautifulSoup
3
4   url = "https://www.onlinegdb.com/"
5
6   response = requests.get(url)
7   soup = Bea (variable) link: Any                    er")
8
9   for link i  See Real World Examples From GitHub
10      print(link.get("href"))
```

```
#editor_1
#
#tab-stdin
#tab-stdout
#tab-stderr
#tab-debug-console
/upgrade-premium
https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/
#debug_window_call_stack
#debug_window_local_variables
#debug_window_registers
#debug_window_display_expression
#debug_window_breakpoints
#
#
```

task_1.py | task_1.java | task_2.c++ | task_3.sql | task_4.py

task_4.js > then() callback

```javascript
fetch("https://www.onlinegdb.com/")
  .then(response => response.text())
  .then(data => {
    const parser = new DOMParser();
    const doc = parser.parseFromString(data, "text/html");

    const links = doc.querySelectorAll("a");
    links.forEach(link => {
      console.log(link.href);
    });
  });
```

```
#editor_1
#
#tab-stdin
#tab-stdout
#tab-stderr
#tab-debug-console
/upgrade-premium
https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/
#debug_window_call_stack
#debug_window_local_variables
#debug_window_registers
#debug_window_display_expression
#debug_window_breakpoints
#
#
#
```

Explanation: In this assignment, we used AI to convert code from one programming language to another. We converted Python to Java, C++ to Python, SQL to Pandas, and also worked on a real-time example.
We checked that all converted programs give correct output. This helped us understand differences between languages and how AI can make coding easier and faster.